TikuOS Bus Interfaces: I2C, SPI & ADC

Simple. Ubiquitous. Intelligence, Everywhere.

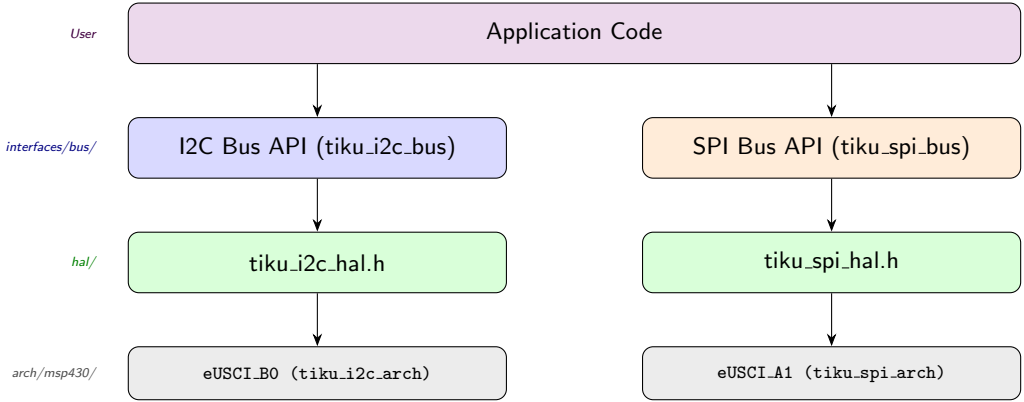
Ambuj Varshney
ambuj@tiku-os.org

March 11, 2026

Outline

1. Architecture Overview
2. I2C Bus Interface
   - I2C Protocol
   - I2C API
   - I2C Architecture
3. SPI Bus Interface
   - SPI Protocol
   - SPI API
   - SPI Architecture
4. ADC Interface
   - ADC Overview
   - ADC API
   - ADC Architecture
5. Usage Examples
6. File Organization
7. Summary

Three-Layer Bus Abstraction



Design Principle

Platform-independent API in `interfaces/bus/`. Board-specific pin routing and clock prescalers in board headers. One arch implementation works across all MSP430FR variants.

I2C vs SPI at a Glance

	I2C	**SPI**
Wires	2 (SDA + SCL)	3 + 1/device (SCLK + SIMO + SOMI + CS)
Addressing	7-bit slave address on bus	Chip select GPIO per device
Duplex	Half-duplex	Full-duplex
Speed	100 kHz / 400 kHz	Up to SMCLK/2 (4 MHz @ 8 MHz)
MSP430 Module	eUSCI_B0	eUSCI_A1
Typical Use	Sensors, EEPROMs, RTCs	Flash, displays, radio transceivers

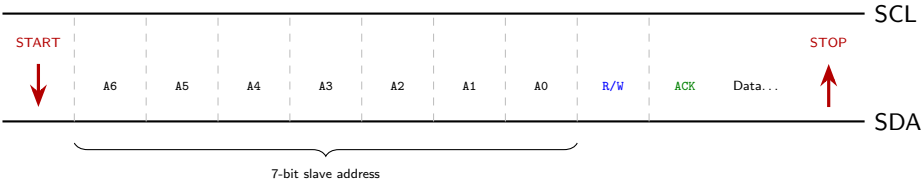
FR5969 LaunchPad I2C Pins

P1.6 = SDA, P1.7 = SCL
(eUSCI_B0, SEL1=1 SEL0=0)

FR5969 LaunchPad SPI Pins

P2.5 = CLK, P2.6 = SIMO, P2.7 = SOMI
(eUSCI_A1, SEL1=1 SEL0=0)

I2C Protocol Basics



Transaction types:

- **Write**: START → addr+W → data → STOP
- **Read**: START → addr+R → data → NACK → STOP
- **Write-Read**: write → repeated START → read (register-read pattern)

Key characteristics:

- Multi-master, multi-slave on same bus
- ACK/NACK feedback per byte
- Clock stretching by slave
- Open-drain lines with pull-ups

I2C Configuration & Status Codes

interfaces/bus/tiku_i2c_bus.h

```
/* Speed modes */
#define TIKU_I2C_SPEED_STANDARD 0
#define TIKU_I2C_SPEED_FAST     1

/* Status codes */
#define TIKU_I2C_OK           0
#define TIKU_I2C_ERR_NACK    (-1)
#define TIKU_I2C_ERR_TIMEOUT (-2)
#define TIKU_I2C_ERR_BUSY    (-3)
#define TIKU_I2C_ERR_PARAM   (-4)

/* Configuration */
typedef struct tiku_i2c_config {
    uint8_t speed;
} tiku_i2c_config_t;
```

Speed modes:

- `STANDARD`: 100 kHz (prescaler 80)
- `FAST`: 400 kHz (prescaler 20)
- Prescalers from 8 MHz SMCLK

Error Handling

Every bus operation returns a status code.
`NACK`: slave didn't respond — wrong address or busy.
`TIMEOUT`: bus stuck — hardware issue.
All errors auto-issue STOP to release bus.

I2C API Functions

Function	Description
`tiku_i2c_init(config)`	Initialize eUSCI_B0 in I2C master mode
`tiku_i2c_close()`	Place peripheral in reset
`tiku_i2c_write(addr, buf, len)`	START → addr+W → data → STOP
`tiku_i2c_read(addr, buf, len)`	START → addr+R → data → NACK → STOP
`tiku_i2c_write_read(addr, tx, tx_len, rx, rx_len)`	Write then repeated-START read (register access)

Typical register-read pattern

```
/* Read 2 bytes from register 0x05 of device at address 0x48 */
uint8_t reg = 0x05;
uint8_t buf[2];
int rc = tiku_i2c_write_read(0x48, &reg, 1, buf, 2);
if (rc != TIKU_I2C_OK) {
    /* handle error */
}
```

I2C Board & Device Macros

devices/tiku_device_fr5969.h

```
/* eUSCI peripheral availability */
#define TIKU_DEVICE_HAS_EUSCIB0 1
#define TIKU_DEVICE_HAS_EUSCIB1 1
```

boards/tiku_board_fr5969_launchpad.h

```
/* I2C on eUSCI_B0:
   P1.6 = SDA, P1.7 = SCL */
#define TIKU_BOARD_I2C_PINS_INIT() \
    do { P1SEL1 |= BIT6 | BIT7; \
         P1SEL0 &= ~(BIT6 | BIT7);\
    } while(0)

/* Prescalers from 8 MHz SMCLK */
#define TIKU_BOARD_I2C_BRW_100K 80
#define TIKU_BOARD_I2C_BRW_400K 20
```

Adding a new board:

1. Create board header in `boards/`
2. Define `TIKU_BOARD_I2C_PINS_INIT()` with correct port and pin SEL bits
3. Define prescaler macros for the board's SMCLK frequency
4. Add `#elif` in `tiku_device_select.h`

Pin Function Select

MSP430FR5969 uses SEL1:SEL0 = 1:0 for eUSCI_B0 alternate function on P1.6/P1.7.

I2C Arch: Initialization

arch/msp430/tiku_i2c_arch.c — tiku_i2c_arch_init()

```
int tiku_i2c_arch_init(const tiku_i2c_config_t *config)
{
    /* Select I2C function on board-specific SDA/SCL pins */
    TIKU_BOARD_I2C_PINS_INIT();

    /* Put eUSCI_B0 in reset before configuration */
    UCB0CTLW0 = UCSWRST;

    /* I2C master mode, synchronous, SMCLK source */
    UCB0CTLW0 |= UCMODE_3 | UCMST | UCSSEL__SMCLK | UCSYNC;

    /* Set clock prescaler for requested speed */
    if (config->speed == TIKU_I2C_SPEED_FAST) {
        UCB0BRW = TIKU_BOARD_I2C_BRW_400K;
    } else {
        UCB0BRW = TIKU_BOARD_I2C_BRW_100K;
    }

    UCB0CTLW1 = UCASTP_0;        /* Manual STOP (no auto-stop) */
    UCB0CTLW0 &= ~UCSWRST;       /* Release from reset */
    return TIKU_I2C_OK;
}
```

I2C Arch: Write & Read

Write transaction

```c
int tiku_i2c_arch_write(
    uint8_t addr,
    const uint8_t *buf, uint16_t len)
{
    UCB0I2CSA = addr;
    UCB0CTLW0 |= UCTR | UCTXSTT;

    for (i = 0; i < len; i++) {
        rc = i2c_wait_flag(UCTXIFG0);
        if (rc != TIKU_I2C_OK)
            return rc;
        UCB0TXBUF = buf[i];
    }

    /* Wait for last byte */
    rc = i2c_wait_flag(UCTXIFG0);
    if (rc != TIKU_I2C_OK)
        return rc;

    UCB0CTLW0 |= UCTXSTP;
    return i2c_wait_stop();
}
```

Read transaction (multi-byte)

```c
int tiku_i2c_arch_read(
    uint8_t addr,
    uint8_t *buf, uint16_t len)
{
    UCB0I2CSA = addr;
    UCB0CTLW0 &= ~UCTR;
    UCB0CTLW0 |= UCTXSTT;

    for (i = 0; i < len; i++) {
        /* STOP before last byte */
        if (i == len - 1)
            UCB0CTLW0 |= UCTXSTP;

        rc = i2c_wait_flag(UCRXIFG0);
        if (rc != TIKU_I2C_OK)
            return rc;
        buf[i] = (uint8_t)UCB0RXBUF;
    }

    return i2c_wait_stop();
}
```

Single-Byte Read

For `len==1`, STOP must be issued right after the address phase completes (special handling in the

I2C Arch: Timeout & NACK Detection

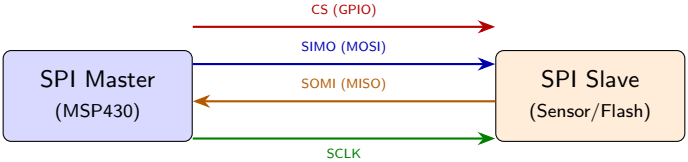
Timeout-guarded flag wait with NACK detection

```
static int i2c_wait_flag(uint16_t flag)
{
    uint16_t timeout = 10000U;

    while (!(UCB0IFG & flag)) {
        if (UCB0IFG & UCNACKIFG) {        /* Slave NACKed */
            UCB0IFG &= ~UCNACKIFG;
            UCB0CTLW0 |= UCTXSTP;         /* Issue STOP to release bus */
            while (UCB0CTLW0 & UCTXSTP);
            return TIKU_I2C_ERR_NACK;
        }
        if (--timeout == 0) {
            UCB0CTLW0 |= UCTXSTP;         /* Force STOP on timeout */
            return TIKU_I2C_ERR_TIMEOUT;
        }
    }
    return TIKU_I2C_OK;
}
```

Bus Recovery

Every error path issues a STOP condition to release SDA/SCL, preventing bus lockup. The timeout

SPI Protocol Basics



Four SPI clock modes:

Mode	CPOL	CPHA	Sampling Edge
0	0	0	Rising (most common)
1	0	1	Falling
2	1	0	Falling
3	1	1	Rising

Key characteristics:

- Full-duplex: TX and RX simultaneous
- No addressing — CS selects device
- No ACK/NACK feedback
- Master always generates clock
- Much faster than I2C

SPI Configuration

interfaces/bus/tiku_spi_bus.h

```
/* Clock modes */
#define TIKU_SPI_MODE_0   0
#define TIKU_SPI_MODE_1   1
#define TIKU_SPI_MODE_2   2
#define TIKU_SPI_MODE_3   3

/* Bit order */
#define TIKU_SPI_MSB_FIRST   0
#define TIKU_SPI_LSB_FIRST   1

/* Status codes */
#define TIKU_SPI_OK            0
#define TIKU_SPI_ERR_TIMEOUT  (-1)
#define TIKU_SPI_ERR_PARAM    (-2)

typedef struct tiku_spi_config {
    uint8_t  mode;
    uint8_t  bit_order;
    uint16_t prescaler;
} tiku_spi_config_t;
```

Prescaler convenience macros:
(from `tiku_board_fr5969_launchpad.h`)

Macro	SPI Clock
TIKU_BOARD_SPI_BRW_4MHZ	4 MHz
TIKU_BOARD_SPI_BRW_2MHZ	2 MHz
TIKU_BOARD_SPI_BRW_1MHZ	1 MHz
TIKU_BOARD_SPI_BRW_500KHZ	500 kHz

All derived from 8 MHz SMCLK:
`SPI_CLK = SMCLK / prescaler`

Chip Select

CS is managed by the application via GPIO, not by the SPI driver. This supports arbitrary CS pin assignment and multi-device buses.

SPI API Functions

Function	Description
`tiku_spi_init(config)`	Initialize eUSCI_A1 in 3-pin SPI master mode
`tiku_spi_close()`	Place peripheral in reset
`tiku_spi_transfer(byte)`	Single-byte full-duplex exchange (returns RX byte)
`tiku_spi_write(buf, len)`	Bulk write; received data is discarded
`tiku_spi_read(buf, len)`	Bulk read; sends 0xFF as dummy data
`tiku_spi_write_read(tx, rx, len)`	Bulk full-duplex simultaneous TX and RX

Typical SPI device interaction

```
/* Assert CS, send command, read response, deassert CS */
P3OUT &= ~BIT0;                    /* CS low (active) */
tiku_spi_write(cmd, sizeof(cmd));  /* Send command bytes */
tiku_spi_read(response, 4);        /* Read 4 bytes (sends 0xFF) */
P3OUT |= BIT0;                     /* CS high (inactive) */
```

SPI Board Macros & Mode Mapping

boards/tiku_board_fr5969_launchpad.h

```
/* SPI on eUSCI_A1:
   P2.5=CLK P2.6=SIMO P2.7=SOMI */
#define TIKU_BOARD_SPI_PINS_INIT() \
    do { \
        P2SEL1 |= BIT5|BIT6|BIT7; \
        P2SEL0 &= ~(BIT5|BIT6|BIT7);\
    } while(0)

/* Prescalers: 8 MHz SMCLK */
#define TIKU_BOARD_SPI_BRW_4MHZ   2
#define TIKU_BOARD_SPI_BRW_2MHZ   4
#define TIKU_BOARD_SPI_BRW_1MHZ   8
#define TIKU_BOARD_SPI_BRW_500KHZ 16
```

MSP430 SPI mode mapping:

MSP430's `UCCKPH` is **inverted** relative to the standard CPHA definition.

SPI Mode	UCCKPL	UCCKPH	Bits
Mode 0	0	1	`UCCKPH`
Mode 1	0	0	0
Mode 2	1	1	`UCCKPH|UCCKPL`
Mode 3	1	0	`UCCKPL`

Common Pitfall

Standard CPHA=0 maps to MSP430 UCCKPH=1. The driver handles this mapping internally — just use `TIKU_SPI_MODE_0..3`.

SPI Arch: Initialization

arch/msp430/tiku_spi_arch.c — tiku_spi_arch_init()

```
int tiku_spi_arch_init(const tiku_spi_config_t *config)
{
    /* Select SPI function on board-specific SCLK/SIMO/SOMI pins */
    TIKU_BOARD_SPI_PINS_INIT();

    /* Put eUSCI_A1 in reset before configuration */
    UCA1CTLW0 = UCSWRST;

    /* SPI master, synchronous, 3-pin mode, SMCLK source */
    UCA1CTLW0 |= UCMST | UCSYNC | UCMODE_0 | UCSSEL__SMCLK;

    /* Set clock polarity and phase for requested SPI mode */
    UCA1CTLW0 |= spi_mode_to_bits(config->mode);

    /* Set bit order */
    if (config->bit_order == TIKU_SPI_MSB_FIRST)
        UCA1CTLW0 |= UCMSB;

    /* Set clock prescaler */
    UCA1BRW = config->prescaler;

    UCA1CTLW0 &= ~UCSWRST;  /* Release from reset */
    return TIKU_SPI_OK;
}
```

SPI Arch: Transfer Operations

Single-byte full-duplex exchange

```c
static uint8_t spi_xfer_byte(uint8_t tx)
{
    uint16_t timeout;

    timeout = SPI_TIMEOUT;
    while (!(UCA1IFG & UCTXIFG)) {
        if (--timeout == 0)
            return 0;
    }
    UCA1TXBUF = tx;

    timeout = SPI_TIMEOUT;
    while (!(UCA1IFG & UCRXIFG)) {
        if (--timeout == 0)
            return 0;
    }
    return (uint8_t)UCA1RXBUF;
}
```

Bulk read (sends 0xFF dummy)

```c
int tiku_spi_arch_read(
    uint8_t *buf, uint16_t len)
{
    uint16_t i, timeout;

    for (i = 0; i < len; i++) {
        timeout = SPI_TIMEOUT;
        while (!(UCA1IFG & UCTXIFG))
            if (--timeout == 0)
                return TIKU_SPI_ERR_TIMEOUT;
        UCA1TXBUF = 0xFF;

        timeout = SPI_TIMEOUT;
        while (!(UCA1IFG & UCRXIFG))
            if (--timeout == 0)
                return TIKU_SPI_ERR_TIMEOUT;
        buf[i] = (uint8_t)UCA1RXBUF;
    }
    return TIKU_SPI_OK;
}
```

SPI is Master-Clocked

Unlike I2C, SPI has no bus contention — the master always generates the clock. Timeouts guard against hardware faults only.

ADC: Reading Analog Sensors

ADC12_B on MSP430FR5x:

- 12-bit SAR ADC (configurable 8/10/12-bit)
- 16 external channels (A0–A15)
- Internal channels: temperature sensor (ch30), battery monitor (ch31)
- Multiple reference sources: AVCC, 1.2V, 2.0V, 2.5V
- Single-channel single-conversion mode

Typical analog sensors:

- LM35/TMP36 — temperature (10 mV/°C)
- Photoresistor (LDR) — light level
- Potentiometer — position/angle
- Force/pressure sensors
- Internal MCU temp sensor (no wiring!)

Same Three-Layer Pattern

`interfaces/adc/` → `hal/` → `arch/msp430/`

ADC Configuration & Status Codes

interfaces/adc/tiku_adc.h

```
/* Resolution */
#define TIKU_ADC_RES_8BIT    0
#define TIKU_ADC_RES_10BIT   1
#define TIKU_ADC_RES_12BIT   2

/* Reference voltage */
#define TIKU_ADC_REF_AVCC    0  /* 3.3V */
#define TIKU_ADC_REF_1V2     1
#define TIKU_ADC_REF_2V0     2
#define TIKU_ADC_REF_2V5     3

/* Status codes */
#define TIKU_ADC_OK           0
#define TIKU_ADC_ERR_TIMEOUT (-1)
#define TIKU_ADC_ERR_PARAM   (-2)

typedef struct tiku_adc_config {
    uint8_t resolution;
    uint8_t reference;
} tiku_adc_config_t;
```

Resolution vs. range:

Bits	Max Value	Step (3.3V)
8	255	12.9 mV
10	1023	3.22 mV
12	4095	0.81 mV

Reference selection:

- `AVCC`: Full 0–3.3V range
- `1V2/2V0/2V5`: Higher precision for low-voltage sensors
- Internal ref uses the REF module

ADC API Functions

Function	Description
`tiku_adc_init(config)`	Configure ADC12_B resolution and reference
`tiku_adc_close()`	Disable ADC and reference (save power)
`tiku_adc_channel_init(ch)`	Set pin to analog mode for external channel
`tiku_adc_read(ch, &value)`	Single-conversion blocking read

Reading an analog sensor on A2 (P1.2)

```
tiku_adc_config_t cfg = {
    .resolution = TIKU_ADC_RES_12BIT,
    .reference  = TIKU_ADC_REF_AVCC
};
uint16_t value;

tiku_adc_init(&cfg);
tiku_adc_channel_init(2);        /* P1.2 = A2 for analog */
tiku_adc_read(2, &value);        /* Raw 12-bit result: 0-4095 */

/* Convert to millivolts: mV = value * 3300 / 4095 */
```

ADC Arch: Initialization

arch/msp430/tiku_adc_arch.c — tiku_adc_arch_init()

```
int tiku_adc_arch_init(const tiku_adc_config_t *config)
{
    ADC12CTL0 &= ~ADC12ENC;                /* Disable before config */
    ADC12CTL0 = ADC12ON | ADC12SHT0_6;     /* On, 128-cycle S&H */

    /* Single-channel, single-conversion, sampling timer, MODCLK */
    ADC12CTL1 = ADC12SHP | ADC12CONSEQ_0 | ADC12SSEL_0;

    /* Resolution: 8, 10, or 12 bits */
    ADC12CTL2 = adc_resolution_bits(config->resolution);

    /* Enable internal channel mapping (temp sensor, battery) */
    ADC12CTL3 = ADC12TCMAP | ADC12BATMAP;

    /* Configure internal reference if not using AVCC */
    adc_reference_bits(config->reference);
    adc_wait_ref_ready();

    return TIKU_ADC_OK;
}
```

Sample-and-Hold Time

ADC Arch: Channel Init & Read

Pin configuration per channel

```c
int tiku_adc_arch_channel_init(
    uint8_t channel)
{
    if (channel >= 30) /* internal */
        return TIKU_ADC_OK;

    /* Set SEL0=SEL1=1 for analog */
    switch (channel) {
    case 0: P1SEL0 |= BIT0;
            P1SEL1 |= BIT0; break;
    case 1: P1SEL0 |= BIT1;
            P1SEL1 |= BIT1; break;
    /* ... A2-A5 on P1.2-P1.5 */
    /* ... A8-A11 on P4.0-P4.3 */
    /* ... A12-A15 on P3.0-P3.3 */
    }
    return TIKU_ADC_OK;
}
```

Single-conversion read

```c
int tiku_adc_arch_read(
    uint8_t ch, uint16_t *value)
{
    uint16_t timeout;

    ADC12CTL0 &= ~ADC12ENC;
    /* Channel + reference select */
    ADC12MCTL0 = vrsel | ch;
    ADC12IFGR0 &= ~ADC12IFG0;

    /* Start conversion */
    ADC12CTL0 |= ADC12ENC | ADC12SC;

    /* Wait for completion */
    timeout = 10000U;
    while (!(ADC12IFGR0 & ADC12IFG0))
        if (--timeout == 0)
            return TIKU_ADC_ERR_TIMEOUT;

    *value = ADC12MEM0;
    return TIKU_ADC_OK;
}
```

Internal Channels

Channel 30 = on-chip temperature sensor.

Channel 31 = (AVCC−AVSS)/2 for battery

FR5969 LaunchPad ADC Channel Map

Channel	Pin	BoosterPack	Note
A0	P1.0	J1.2	Conflicts with LED2
A1	P1.1	J2.19	Conflicts with Button S2
A2	P1.2	J1.4	Available
A3	P1.3	J1.5	Available
A4	P1.4	J1.6	Available
A5	P1.5	J1.7	Available
A12	P3.0	J2.15	Available
A13	P3.1	J2.14	Available
A14	P3.2	J2.13	Available
A15	P3.3	J2.12	Available
30	Internal temp sensor		Use with 1.2V ref
31	(AVCC–AVSS)/2		Battery monitoring

Pin Analog Mode

Set `PxSEL0|=BITn; PxSEL1|=BITn;` (SEL1:SEL0 = 1:1) for analog input.

Example: I2C Temperature Sensor (Compile-Time Selection)

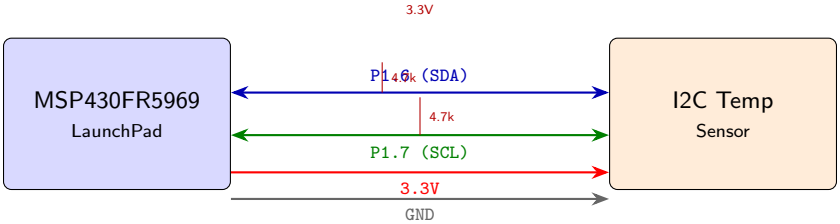
Sensor-specific constants selected by #define in tiku_example_config.h

```
/* tiku_example_config.h --- select ONE sensor */
#define TIKU_TEMP_SENSOR_MCP9808  0  /* Microchip MCP9808 (addr 0x18) */
#define TIKU_TEMP_SENSOR_ADT7410  1  /* Analog Devices ADT7410 (addr 0x48) */

/* i2c_temp.c --- sensor-specific constants */
#if TIKU_TEMP_SENSOR_MCP9808
  #define SENSOR_NAME   "MCP9808"
  #define SENSOR_ADDR   0x18
  #define SENSOR_REG_TEMP  0x05   /* 13-bit signed, 0.0625 C/LSB */
#elif TIKU_TEMP_SENSOR_ADT7410
  #define SENSOR_NAME   "ADT7410"
  #define SENSOR_ADDR   0x48
  #define SENSOR_REG_TEMP  0x00   /* 13-bit signed, 0.0625 C/LSB */
#endif

/* Common: read 16-bit big-endian register (works for both sensors) */
static int sensor_read_reg16(uint8_t reg, uint16_t *value)
{
    uint8_t buf[2];
    int rc = tiku_i2c_write_read(SENSOR_ADDR, &reg, 1, buf, 2);
    if (rc == TIKU_I2C_OK)
        *value = ((uint16_t)buf[0] << 8) | buf[1];
    return rc;
}
```

Adding a New Sensor

Define `SENSOR_NAME`, `SENSOR_ADDR`, register addresses, and add an `#elif` block for ID verification and

Sensor Wiring: MSP430FR5969 LaunchPad



Common I2C Connections

LaunchPad	Sensor
P1.6 (J1 pin 10)	SDA
P1.7 (J1 pin 9)	SCL
3.3V	VDD
GND	GND + addr pins

Supported Sensors

Sensor	Addr	Addr Pins
MCP9808	0x18	A0,A1,A2→GND
ADT7410	0x48	A0,A1→GND

Pull-ups: 4.7 kΩ on SDA/SCL to 3.3V (most breakouts include these). Same wiring for both sensors.

MCP9808 vs ADT7410: Register Comparison

	MCP9808 (Microchip)	**ADT7410 (Analog Devices)**
I2C Address	0x18 (A0=A1=A2=GND)	0x48 (A0=A1=GND)
Temp Register	0x05	0x00
Resolution	0.0625 °C (13-bit)	0.0625 °C (13-bit default)
Range	−40 to +125 °C	−40 to +150 °C
ID Register	0x06=0x0054 (mfr)	0x0B=0xCB (8-bit)
	0x07=0x04xx (dev)	upper 5 bits = mfr ID
Supply	2.7–5.5 V	2.7–5.5 V

MCP9808 Temp Format

Upper byte: [7:5] alerts, [4] sign, [3:0] int
Lower byte: [7:4] int, [3:0] frac (1/16 °C)
Sign bit in upper byte bit 4

ADT7410 Temp Format

16-bit register, bits [15:3] = temperature
Bits [2:0] = status flags
Bit 15 = sign (two's complement)

Same Resolution

Both sensors use 0.0625 °C per LSB, so the fractional display logic is identical — only parsing differs.

Example: SPI Flash Read

SPI flash read — W25Q SPI flash memory

```
#define FLASH_CS_LOW()   do { P3OUT &= ~BIT0; } while(0)
#define FLASH_CS_HIGH()  do { P3OUT |=  BIT0; } while(0)
#define FLASH_CMD_READ   0x03

static void flash_read(uint32_t addr, uint8_t *buf, uint16_t len)
{
    tiku_spi_config_t cfg = {
        .mode      = TIKU_SPI_MODE_0,
        .bit_order = TIKU_SPI_MSB_FIRST,
        .prescaler = TIKU_BOARD_SPI_BRW_2MHZ
    };
    uint8_t cmd[4];

    tiku_spi_init(&cfg);

    /* Build 4-byte read command: opcode + 24-bit address */
    cmd[0] = FLASH_CMD_READ;
    cmd[1] = (uint8_t)(addr >> 16);
    cmd[2] = (uint8_t)(addr >> 8);
    cmd[3] = (uint8_t)(addr);

    FLASH_CS_LOW();
    tiku_spi_write(cmd, 4);       /* Send command + address */
    tiku_spi_read(buf, len);      /* Clock out data (sends 0xFF) */
```

Example 10: I2C Temp Sensor Process (examples/10_i2c_temp/)

Periodic I2C temperature read — sensor selected at compile time

```
static struct tiku_timer poll_timer;
TIKU_PROCESS(temp_sensor_process, SENSOR_NAME);  /* "MCP9808" or "ADT7410" */

TIKU_PROCESS_THREAD(temp_sensor_process, ev, data)
{
    static tiku_i2c_config_t i2c_cfg;
    static int16_t temp_int;
    static uint8_t temp_frac, temp_neg;

    TIKU_PROCESS_BEGIN();
    tiku_common_led1_init();
    tiku_common_led2_init();

    /* Init I2C at 100 kHz, verify sensor identity */
    i2c_cfg.speed = TIKU_I2C_SPEED_STANDARD;
    tiku_i2c_init(&i2c_cfg);
    if (!sensor_verify_id()) {      /* Checks MCP9808 or ADT7410 IDs */
        tiku_i2c_close();
        TIKU_PROCESS_EXIT();
    }

    tiku_timer_set_event(&poll_timer, 2 * TIKU_CLOCK_SECOND);

    while (1) {
        TIKU_PROCESS_WAIT_EVENT_UNTIL(ev == TIKU_EVENT_TIMER);

        if (sensor_read_temp(&temp_int, &temp_frac, &temp_neg)
            == TIKU_I2C_OK) {
            int frac = (int)((uint16_t)temp_frac * 625 / 100);
            TIKU_PRINTF("[%s] Temp: %s%d.%d C\n", SENSOR_NAME,
                        temp_neg ? "-" : "", (int)temp_int, frac);
            tiku_common_led1_toggle();
```

Bus Interface Module Files

Layer	File	Role
I2C API	`interfaces/bus/tiku_i2c_bus.h` `interfaces/bus/tiku_i2c_bus.c`	Public types & prototypes Validation, delegates to arch
SPI API	`interfaces/bus/tiku_spi_bus.h` `interfaces/bus/tiku_spi_bus.c`	Public types & prototypes Validation, delegates to arch
ADC API	`interfaces/adc/tiku_adc.h` `interfaces/adc/tiku_adc.c`	Public types & prototypes Validation, delegates to arch

HAL	`hal/tiku_i2c_hal.h`
	`hal/tiku_spi_hal.h`
	`hal/tiku_adc_hal.h`
		Routes to I2C arch
		Routes to SPI arch
		Routes to ADC arch

Arch (MSP430)	`arch/msp430/tiku_i2c_arch.c/.h`
	`arch/msp430/tiku_spi_arch.c/.h`
	`arch/msp430/tiku_adc_arch.c/.h`
		eUSCI_B0 I2C master
		eUSCI_A1 SPI master
		ADC12_B analog input

Board Config	`.../tiku_device_fr5969.h` `.../tiku_board_fr5969_launchpad.h`	eUSCI availability flags Pin select & prescaler macros
Example	`examples/10_i2c_temp/`	I2C temp sensor (MCP9808/ADT7410)

Summary

I2C Bus

- 2-wire (SDA + SCL) via eUSCI_B0
- 100 kHz standard / 400 kHz fast
- 3 operations: write, read, write_read
- NACK detection + timeout guards
- Auto STOP on every error path

Architecture

- Platform-independent API in `interfaces/`
- HAL routing in `hal/`
- MSP430 arch in `arch/msp430/`
- Board macros for pin config & clock
- Same arch code works across all FR variants

SPI Bus

- 3-wire + CS via eUSCI_A1
- All 4 SPI modes, MSB/LSB order
- 4 operations: transfer, write, read, write_read
- Application-managed CS via GPIO
- Up to 4 MHz from 8 MHz SMCLK

FR5969 LaunchPad Module Assignment

eUSCI_A0	UART (backchannel)
eUSCI_A1	SPI
eUSCI_B0	I2C
eUSCI_B1	(available)
ADC12_B	Analog input

Thank You

TikuOS Bus Interfaces

I2C, SPI & ADC for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
I2C API:	`interfaces/bus/tiku_i2c_bus.h`
SPI API:	`interfaces/bus/tiku_spi_bus.h`
ADC API:	`interfaces/adc/tiku_adc.h`
I2C Arch:	`arch/msp430/tiku_i2c_arch.c`
SPI Arch:	`arch/msp430/tiku_spi_arch.c`
ADC Arch:	`arch/msp430/tiku_adc_arch.c`